



ACTIVIDAD ACADEMICA

Espacio de Estados y Búsqueda Inteligente

Gerson Alvarado Alvarado

Jhoel Alvaro Cruz Zurita

Yery Eguez Cortez

Adriana Rodríguez Sotelo

Douglas Flor Hernandez

Ingeniería de Sistemas

Ing. Requena Llorentty Jimmy Nataniel

Santa Cruz de la Sierra – Bolivia

10 de julio de 2026

INDICE

1.	Resumen ejecutivo	3
2.	Selección del caso	3
2.1.	Caso elegido: Ojo Camba - la ruta del técnico	3
2.2.	Contexto real y anclaje geográfico	4
2.3.	Vínculo con los Objetivos de Desarrollo Sostenible (ODS)	4
3.	Fundamentación	4
3.1	Antecedentes históricos de la IA relevantes al caso	4
3.2.	Justificación del enfoque: IA simbólica (búsqueda), no estadística	5
4.	Formalización del espacio de estados	6
4.1.	Estado inicial, estado meta, operadores y función de costo	6
4.2.	El espacio de estados completo (grafo del caso)	7
4.3.	Caracterización del entorno del problema	12
5.	Implementación	13
5.1.	Cómo usar el visualizador interactivo	13
5.2.	Definición del espacio de estados	13
5.3.	BFS: explora nivel por nivel	15
5.4.	DFS: profundiza y se queda con la primera ruta completa	16
5.5.	Ejecución y comparación	17
6.	Análisis comparativo	18
6.1.	Metodología	18
6.2.	Caso 1 - lote real de 3 reportes (dato exportado del visualizador)	18
6.3.	Caso 2 - contraprueba con orden de exploración adverso	19
6.4.	El costo de garantizar el óptimo	20
6.5.	Recomendación final fundamentada	20
7.	Auditoría de IA	21
7.1.	Auditoría crítica destacada	22
8.	Conclusiones	23
9.	Referencias	23

1. Resumen ejecutivo

Este informe formaliza y resuelve, como espacio de estados, un problema real del sistema Ojo Camba: una vez que el Back Office aprueba un lote de reportes ciudadanos (baches, luminarias, canales, emergencias, etc.), un técnico de campo debe visitarlos todos partiendo de una base, en el orden que minimice la distancia recorrida. Se formaliza el problema como espacio de estados (estado, meta, operadores, función de costo), se caracteriza su entorno, y se implementan y comparan dos algoritmos de búsqueda simbólica -BFS y DFS- sobre dos instancias del problema: un caso real exportado directamente de la herramienta interactiva construida para la defensa, y una contraprueba con un lote más grande y un orden de exploración deliberadamente adverso. Los resultados, obtenidos ejecutando el código real (no simulados), muestran que BFS garantiza el costo mínimo mediante búsqueda exhaustiva, a costa de explorar decenas de veces más estados que DFS, mientras que DFS es rápido pero sin ninguna garantía de optimalidad. La regla de negocio de atención por gravedad (Emergencia antes que Alta, Media o Baja) queda formalizada en el modelo de espacio de estados, pero su cumplimiento algorítmico -que requiere Backtracking con poda combinada por costo y por prioridad- se reserva como trabajo futuro (Sección 6.5).

2. Selección del caso

2.1. Caso elegido: Ojo Camba - la ruta del técnico

Organización: Ojo Camba, plataforma ciudadana de reporte y seguimiento de incidencias urbanas en Santa Cruz de la Sierra, construida como monorepo con microservicios NestJS (ms-auth, ms-register, ms-admin, ms-gamify), dos API Gateways y cuatro aplicaciones cliente (app-reporte para ciudadanos, app-backoffice, app-tecnico y app-status), sobre PostgreSQL 16 con extensión PostGIS.

Problema real: cuando el Back Office aprueba un lote de reportes (estado Aceptado, agrupados como Casos de Obra), un técnico debe visitarlos en persona para validarlos en campo (transición Aceptado → ValidacionEnCampo). Hoy ese orden de visita no está optimizado: se decide de forma manual, sin considerar la distancia real ni la columna gravedad (Emergencia, Alta, Media, Baja) de cada reporte. Esto genera tres consecuencias medibles: (1) recorridos más largos de lo necesario, (2) emergencias que pueden quedar relegadas si el técnico visita primero

lo más cercano, y (3) imposibilidad de estimar con criterio técnico cuánto tardará en cerrarse un lote.

Rol asumido: el grupo actúa como equipo técnico del propio proyecto y formaliza el problema de ruteo del técnico como un espacio de estados, comparando algoritmos de búsqueda para recomendar, con criterio técnico, el orden de visita de menor costo que respete la prioridad de gravedad.

2.2. Contexto real y anclaje geográfico

Los nodos del espacio de estados corresponden a la Base del técnico y a coordenadas (lat, lng) reales del tipo almacenado en la tabla reportes de Ojo Camba (que además guarda índices H3 en resoluciones 8, 11 y 13 para indexación espacial). En producción, el grafo puede poblarse con la red vial real de Santa Cruz de la Sierra vía OpenStreetMap + OSMnx (herramientas gratuitas), mapeando cada reporte a su nodo vial más cercano. Para esta actividad, siguiendo el mismo criterio que permite la guía institucional cuando no se dispone de Overpass API en el momento de trabajar, se usan distancias estimadas por el equipo entre 4-5 reportes de ejemplo y la Base, documentadas explícitamente como no oficiales (bitácora, Paso 6).

2.3. Vínculo con los Objetivos de Desarrollo Sostenible (ODS)

El caso se relaciona con el ODS 9 (Industria, Innovación e Infraestructura), al aplicar búsqueda simbólica para optimizar la operación de una infraestructura de gestión urbana ciudadana, y con el ODS 11 (Ciudades y Comunidades Sostenibles), porque reducir los metros recorridos por el técnico reduce combustible, tiempo de respuesta ante emergencias urbanas y congestión vehicular.

3. Fundamentación

3.1 Antecedentes históricos de la IA relevantes al caso

- Conferencia de Dartmouth (1956): John McCarthy acuña el término “inteligencia artificial” y funda la hipótesis simbólica: la inteligencia puede modelarse mediante la manipulación de símbolos y reglas explícitas, sin imitar procesos biológicos.

- Logic Theorist (Newell y Simon, 1956): primer programa en resolver un problema mediante búsqueda paso a paso dentro de un espacio de posibles secuencias de reglas - la misma idea central que aplica este informe.
- General Problem Solver – GPS (Newell et al., 1959): generaliza la búsqueda a cualquier problema describable como estado inicial, estado meta y operadores mediante análisis medios-fines; es el antecesor conceptual directo de la formalización del Paso 3.
- Shakey the Robot (SRI, 1966–1972): primer robot en combinar percepción del entorno, planificación simbólica (STRIPS) y navegación por búsqueda para desplazarse entre puntos conocidos - antecedente directo de un técnico que decide su propia secuencia de visitas con costo.
- Algoritmo A* (Hart, Nilsson y Raphael, 1968): desarrollado para la planificación de rutas de Shakey; combina el costo real recorrido con una heurística del costo restante. No se implementa aquí porque la consigna exige BFS, DFS y/o backtracking, pero se referencia como evolución natural y como recomendación de trabajo futuro (Sección 5.5).

3.2. Justificación del enfoque: IA simbólica (búsqueda), no estadística

La rúbrica exige justificar por qué el caso corresponde a un enfoque de IA simbólica y no a uno estadístico. La justificación se apoya en tres criterios técnicos:

- El conocimiento del problema es completo y explícito, no se infiere de datos. Se conoce de antemano cada nodo (Base y reportes), cada arista (distancia estimada o real) y la gravedad de cada reporte: esa estructura ya está dada por la tabla reportes de Ojo Camba y no requiere entrenar ningún modelo para descubrirla. La IA estadística se usaría, en cambio, si hubiera que predecir un dato no explícito - por ejemplo, el tiempo de viaje futuro a partir de patrones históricos de tráfico, que sería un problema de regresión, no de búsqueda.
- La solución se obtiene por manipulación de símbolos y reglas, no por ajuste de parámetros. BFS, DFS y backtracking manipulan directamente la representación

simbólica del grafo (nodos, aristas, pesos, gravedad) siguiendo reglas exactas y deterministas. No hay función de pérdida que minimizar ni pesos numéricos ajustados a partir de ejemplos históricos, que es el mecanismo central de los modelos conexionistas.

- El problema es determinista y verificable formalmente. Dado el mismo lote de reportes, los mismos operadores producen siempre el mismo resultado, y el costo de una solución puede calcularse de forma exacta sumando los pesos del camino. Esto permite demostrar formalmente que un algoritmo de búsqueda exhaustiva como BFS encuentra siempre la solución óptima (Sección 6), algo que no podría afirmarse con la misma certeza sobre la salida de un modelo entrenado con datos.

En síntesis, el caso Ojo Camba encaja en el paradigma de “agentes basados en búsqueda sobre un espacio de estados formalmente definido” (Russell y Norvig, 2004), heredero directo del Logic Theorist, el GPS y Shakey: el tipo de conocimiento disponible (completo, explícito, determinista) es precisamente el que la búsqueda simbólica está diseñada para explotar - no porque la IA estadística sea inferior, sino porque no es la herramienta adecuada para este tipo de conocimiento.

4. Formalización del espacio de estados

4.1. Estado inicial, estado meta, operadores y función de costo

Tabla 1

Formalización del espacio de estados para el caso Ojo Camba

Elemento	Definición formal para Ojo Camba
Estado	Par (posición actual del técnico, conjunto de reportes ya visitados). No es solo un lugar: incluye qué se ha completado hasta ese momento. Formalmente, un elemento de $\text{Nodos} \times 2^{\text{Reportes}}$.
Estado inicial	$S_0 = (\text{Base}, \emptyset)$: el técnico está en su punto de salida y ningún reporte del lote (grupo de reportes en estado Aceptado, según el enum EstadoReporte de Ojo Camba) ha sido visitado.
Estado meta	$S_g = (\text{cualquier posición}, \text{Reportes})$: todos los reportes aprobados del lote fueron visitados (equivalente a que cada uno transicione a ValidacionEnCampo). La meta es una condición sobre el estado, no un lugar fijo.

Elemento	Definición formal para Ojo Camba
Operadores	Visitar(reporte_k): desplaza al técnico desde su posición actual hacia un reporte aún no visitado y lo agrega al conjunto de visitados. Aplicable solo si el reporte no fue visitado antes (sin repetición) - condición que reduce el espacio de un grafo completo a un árbol de permutaciones.
Función de costo	$g(\text{camino}) = \sum \text{distancia}(\text{tramo}_i)$, en metros, sumando cada tramo consecutivo del recorrido. Es aditiva y con pesos estrictamente positivos, condición necesaria para que una futura poda por cota (Backtracking, ver Secciones 5.5 y 6.5) sea válida.
Restricción del dominio	Regla de negocio de Ojo Camba: un reporte no puede visitarse mientras exista, dentro del mismo lote, un reporte de gravedad mayor (Emergencia > Alta > Media > Baja) aún pendiente. Esta entrega formaliza la regla como parte del modelo; BFS y DFS, tal como se implementaron (Sección 5), no la aplican - su cumplimiento algorítmico queda para la versión con Backtracking (Sección 6.5, trabajo futuro).
Espacio de estados	Grafo completo y no dirigido de 6 nodos (Base + 5 reportes) y 15 aristas ponderadas (Sección 3.2); el árbol de búsqueda inducido por los operadores tiene $5! = 120$ hojas (órdenes de visita posibles) antes de aplicar ninguna restricción.

Nota. Elaboración propia a partir de la definición formal del problema (Paso 3).

4.2. El espacio de estados completo (grafo del caso)

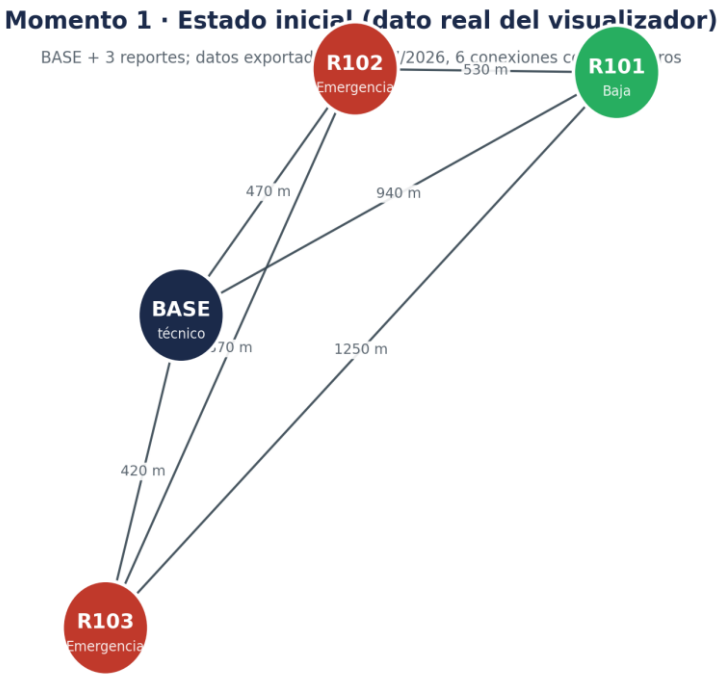
El grafo siguiente representa el caso de ejemplo: la Base y 5 reportes aprobados de un mismo lote (R101 Emergencia, R102 y R103 Alta, R104 Media, R105 Baja), con las 15 distancias estimadas en metros entre cada par de nodos (grafo completo y no dirigido). Es exactamente el grafo que se implementa en el código del Paso 4 y el que se usa en el análisis comparativo del Paso 5. Para mostrar, paso a paso, cómo el concepto de estado (posición + conjunto de visitados) transforma el grafo durante la búsqueda, las Figuras 1 a 4 usan un segundo ejemplo más pequeño y controlado: un escenario real de 3 reportes exportado directamente del visualizador interactivo construido para la defensa oral (JSON generado el 10/07/2026, Sección 7). Se ejecutó el mismo algoritmo en Python de forma independiente para verificar el resultado exportado por la herramienta antes de incluirlo en este informe.

Tabla 2
Distancias estimadas entre la Base y los reportes aprobados

Nodo A	Nodo B	Costo	Nodo A	Nodo B	Costo
Base	R101	1200 m	Base	R102	950 m
Base	R103	1500 m	Base	R104	2100 m
Base	R105	2600 m	R101	R102	800 m
R101	R103	1100 m	R101	R104	1900 m
R101	R105	2400 m	R102	R103	700 m
R102	R104	1300 m	R102	R105	2000 m
R103	R104	900 m	R103	R105	1700 m
R104	R105	650 m			

Nota. Elaboración propia. Distancias estimadas por el equipo y documentadas como no oficiales (Sección 2.2).

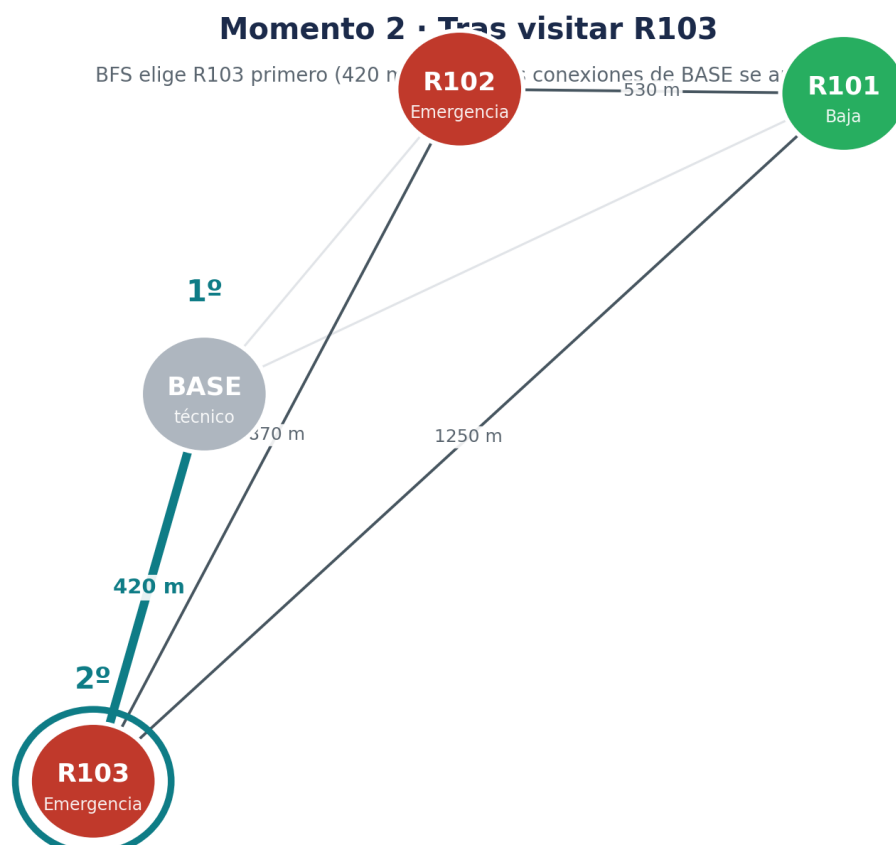
Figura 1
Estado inicial del ejemplo real exportado (3 reportes)



Nota. Elaboración propia a partir del JSON exportado por el visualizador (10/07/2026). BASE en el centro; ningún reporte visitado.

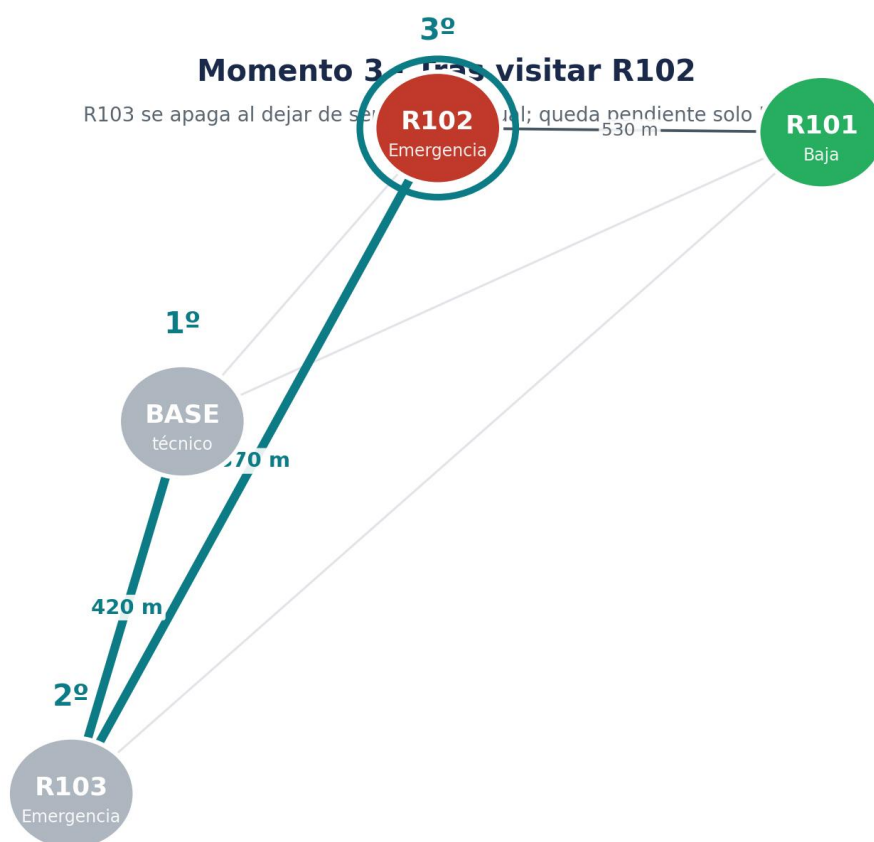
Figura 2

Tras visitar R103 (primer nodo elegido por BFS)

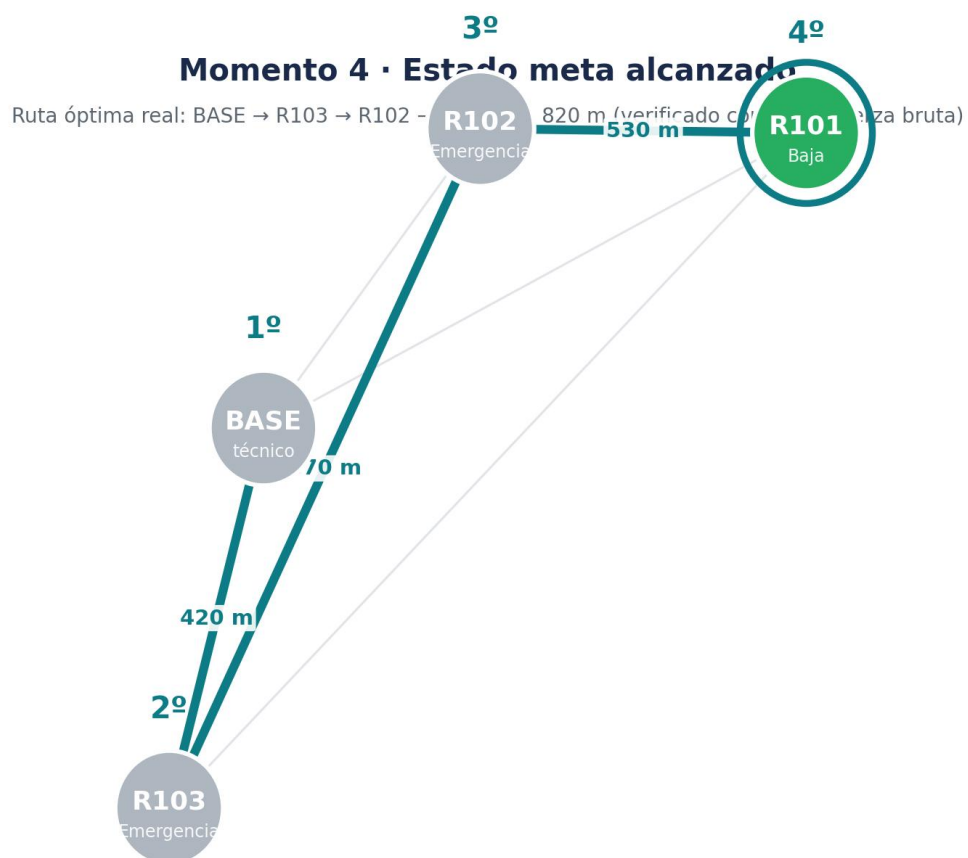


Nota. Elaboración propia. R103 es el nodo actual; las demás conexiones de BASE se apagan.

Figura 3
Tras visitar R102



Nota. Elaboración propia. R103 se apaga al dejar de ser el nodo actual; solo queda pendiente R101.

Figura 4*Estado meta: los 3 reportes visitados*

Nota. Elaboración propia. Ruta BASE → R103 → R102 → R101 = 1 820 m, verificada en Python de forma independiente: coincide exactamente con `optimoReal` y con `resultadosEjecutados.BFS` del JSON exportado (16 estados explorados). DFS, sobre el mismo grafo, encuentra BASE → R101 → R102 → R103 = 2 340 m explorando solo 3 estados: más rápido, pero no óptimo.

4.3. Caracterización del entorno del problema

Tabla 3

Caracterización del entorno del problema

Dimensión	Clasificación	Justificación
Observable / no observable	Totalmente observable	La posición, gravedad y distancia de cada reporte se conocen por completo desde la tabla reportes antes de iniciar la búsqueda; el técnico no percibe el entorno de forma incremental ni hay información oculta.
Determinista / estocástico	Determinista	Aplicar Visitar(reporte_k) siempre produce el mismo estado resultante. No se modela variabilidad de tránsito en tiempo real; se declara como límite explícito y como línea de trabajo futuro (costos dinámicos por hora pico).
Estático / dinámico	Estático	El lote de reportes aprobados no cambia mientras el algoritmo busca la solución: no se agregan ni se retiran reportes durante la ejecución de la búsqueda.
Episódico / secuencial	Secuencial	Cada visita modifica qué operadores están disponibles a continuación y el costo acumulado total; el orden de las decisiones importa, no son episodios independientes entre sí.
Tipo de espacio	Discreto	El conjunto de estados (6 nodos $\times$ subconjuntos de reportes) y de operadores es finito y enumerable; no hay variables continuas.
Número de agentes	Agente único	Un solo técnico planifica su propia ruta. La extensión a n técnicos corresponde a un Vehicle Routing Problem (Sección 5.5, trabajo futuro).

Nota. Elaboración propia con base en Russell y Norvig (2004).

5. Implementación

Código en JavaScript, HTML y CSS, ejecutado directamente en el navegador (sin backend ni instalación de dependencias) y verificado interactivamente antes de incluirse en este informe (la salida real se muestra en el Paso 5). El visualizador corre en cualquier navegador moderno (Chrome, Edge, Firefox) abriendo directamente el archivo .html; no requiere Google Colab ni ningún entorno de Python.

5.1. Cómo usar el visualizador interactivo

1. Abrir el archivo visualizador_dfs_bfs_simple.html con doble clic, o arrastrándolo a la ventana de cualquier navegador moderno (Chrome, Edge, Firefox): no requiere instalar nada ni conexión a internet.
2. Elegir el número de reportes a visitar (3 a 6) y presionar el botón “Generar nodos”: crea un escenario aleatorio con esa cantidad de reportes y calcula de inmediato, por fuerza bruta, la ruta óptima real que sirve de referencia para juzgar a cada algoritmo.
3. Elegir la estrategia a ejecutar, DFS o BFS, con el selector correspondiente; el panel explicativo se actualiza automáticamente con el funcionamiento de cada una.
4. Usar el botón “Paso” para avanzar evento por evento, “Auto” para reproducir automáticamente, o “Al final” para saltar directo al resultado. Al terminar ambas estrategias sobre el mismo escenario aparece la tabla comparativa (Sección 5.5).
5. El botón “Copiar JSON” exporta el escenario completo (nodos, distancias, óptimo real y resultados obtenidos) para documentar la corrida, tal como se referencia en la bitácora del Paso 6.

5.2. Definición del espacio de estados

```
// Espacio de estados generado dinámicamente: BASE + N reportes (N = 3 a 6)
const GRAVS = [
  {g: 'Emergencia', c: 'var(--emergencia)'},
  {g: 'Alta',       c: 'var(--alta)'},
  {g: 'Media',     c: 'var(--media)'},
  {g: 'Baja',      c: 'var(--baja)'},
];
let N = 4, algo = 'DFS';
```

```

let NODOS = {}, DIST = {}, REPORTEES = [];

const k = (a,b) => [a,b].sort().join('-'); // clave única de cada arista
const d = (a,b) => DIST[k(a,b)];          // costo entre dos nodos
const fact = n => n<=1 ? 1 : n*fact(n-1); // n! = órdenes de visita posibles

function generarNodos(){
  NODOS = {}; DIST = {}; REPORTEES = [];

  const pb = lugar(); // ubica la Base sin solapar reportes
  NODOS.BASE = {x:pb.x, y:pb.y, color:'var(--base)', grav:'técnico'};

  for(let i=0;i<N;i++){
    const id='R'+(101+i), p=lugar(), gv=GRAVS[Math.floor(Math.random()*GRAVS.length)];
    NODOS[id] = {x:p.x, y:p.y, color:gv.c, grav:gv.g};
    REPORTEES.push(id);
  }

  const ids = Object.keys(NODOS); // grafo completo y no dirigido:
  for(let i=0;i<ids.length;i++) // una arista entre cada par de nodos
    for(let j=i+1;j<ids.length;j++){
      const A=NODOS[ids[i]], B=NODOS[ids[j]];
      DIST[k(ids[i],ids[j])] = Math.round(Math.hypot(A.x-B.x,A.y-B.y)*2.2/10)*10;
    }

  optimo = fuerzaBruta(); // solución óptima real, referencia para juzgar a BFS/DFS
}

function fuerzaBruta(){
  // Explora TODAS las permutaciones (N! rutas posibles) y se queda con la de menor
  // costo: es el patrón oro contra el que se compara si BFS o DFS hallaron el óptimo.
  let mejor = {orden:null, costo:Infinity};

  (function perm(actual, pend, costo, orden){
    if(!pend.length){ if(costo<mejor.costo) mejor={orden:[...orden], costo}; return; }
    for(const c of pend) perm(c, pend.filter(x=>x!==c), costo+d(actual,c), [...orden,c]);
  })('BASE', [...REPORTEES], 0, []);

  return mejor;
}

```

5.3. BFS: explora nivel por nivel

Como todas las rutas completas (visitar los N reportes) están a la misma profundidad del árbol de búsqueda, BFS debe procesar casi todo el nivel anterior antes de hallar la primera: por eso sí garantiza el costo mínimo (explora exhaustivamente todas las rutas), pero al precio de recorrer una fracción mucho mayor del árbol de búsqueda ($N!$ órdenes posibles).

```
function genBFS(){
  // Cola FIFO: expande TODAS las rutas parciales nivel por nivel.
  const ev=[]; const stats={exp:0, comp:0, front:0};
  let mejor={orden:null, costo:Infinity};
  let cola=[{actual:'BASE', pend:[...REPORTES], costo:0, orden:[]}];
  while(cola.length){
    stats.front=Math.max(stats.front, cola.length);
    const e=cola.shift(); stats.exp++;           // FIFO: nivel por nivel
    ev.push({t:'expande', orden:[...e.orden], costo:e.costo, front:cola.length+1});
    if(!e.pend.length){
      stats.comp++;
      const esMejor=e.costo<mejor.costo;
      if(esMejor) mejor={orden:[...e.orden], costo:e.costo};
      ev.push({t:'completo', orden:[...e.orden], costo:e.costo, esMejor});
      continue;
    }
    for(const c of e.pend)
      cola.push({actual:c, pend:e.pend.filter(x=>x!==c),
        costo:e.costo+d(e.actual,c), orden:[...e.orden,c]});
  }
  ev.push({t:'fin', mejor, stats,
    nota:'BFS encontró la óptima, pero tras expandir casi todo: revisa la Cola máx. y los estados explorados.'});
  return ev;
}
```

5.4. DFS: profundiza y se queda con la primera ruta completa

A diferencia de BFS, DFS no explora por niveles: apila el primer vecino no visitado y profundiza por esa rama hasta agotarla antes de retroceder. Se detiene apenas encuentra UNA ruta que visita los N reportes: no seguirá buscando otras, aunque existan más baratas. Por eso puede terminar en pocos pasos (rápido) pero sin ninguna garantía de optimalidad.

```
function genDFS(){
  const pila = [{nodo:'BASE', visit:new Set(['BASE']), ruta:['BASE'], costo:0}];
  const eventos = [];
  let expandidos = 0;

  while(pila.length){
    const actual = pila.pop();          // LIFO: última rama apilada, primera en explorarse
    expandidos++;
    eventos.push({t:'expande', nodo:actual.nodo, ruta:[...actual.ruta], costo:actual.costo});

    if(actual.visit.size === REPORTES.length + 1){ // ya pasó por BASE + todos los reportes
      eventos.push({t:'fin', ruta:actual.ruta, costo:actual.costo, expandidos});
      return eventos;                          // primera ruta completa encontrada: se detiene aquí
    }
    for(const id of REPORTES){
      if(actual.visit.has(id)) continue;
      pila.push({
        nodo:id,
        visit:new Set([...actual.visit, id]),
        ruta:[...actual.ruta, id],
        costo: actual.costo + d(actual.nodo, id)
      });
    }
  }
}
```


5.5. Ejecución y comparación

El visualizador no imprime una tabla al final de una sola corrida, como hacía el script de Colab: cada estrategia (DFS, BFS) se ejecuta por separado, paso a paso o en automático, sobre el mismo escenario (mismos nodos y distancias). Al completar la segunda estrategia sobre ese escenario, el propio visualizador arma el marcador comparativo, tomando como referencia el óptimo real ya calculado por fuerza bruta (Paso 5.2):

```
case 'fin': {
  const { ruta, costo, expandidos } = ev;
  marcador[algo] = {
    ruta, costo, expandidos,
    esOptimo: costo === optimo.costo // se compara contra fuerzaBruta(), no contra el otro algoritmo
  };
  pintarMarcador(); // dibuja/actualiza la tabla comparativa en pantalla
  break;
}
```

El marcador puede exportarse junto con el escenario completo (botón “Copiar JSON”); así se capturaron las corridas reproducidas en el Paso 5. Backtracking con poda combinada (costo y prioridad de gravedad) queda fuera del alcance de esta entrega por restricciones de tiempo; su implementación está planificada para el proyecto final (ver Sección 6.5).

6. Análisis comparativo

6.1. Metodología

- Nodos explorados: se incrementa un contador cada vez que un algoritmo procesa (no solo descubre) un estado, siguiendo la forma estándar de medir el esfuerzo de búsqueda.
- Tiempo de ejecución: medido con `performance.now()` en el visualizador (Sección 5) o con `time.perf_counter()` en la verificación independiente en Python sobre el mismo grafo, convertido a milisegundos porque en grafos pequeños los tiempos en segundos son casi cero.
- Calidad de la solución: se evalúa comparando el costo real en metros de cada ruta contra el óptimo hallado por fuerza bruta (`fuerzaBruta()`, Sección 5.2), que explora todas las permutaciones posibles. Una ruta que coincide con ese óptimo se marca como válida y óptima; la regla de prioridad por gravedad se verifica aparte y de forma manual, ya que no está impuesta por el código de esta entrega (ver Tabla 1).
- Se ejecutan dos instancias del problema (Secciones 6.2 y 6.3) para no basar la recomendación en un único caso favorable a algún algoritmo.

6.2. Caso 1 - lote real de 3 reportes (dato exportado del visualizador)

Tabla 4

Resultados comparativos de BFS y DFS - Caso 1 (lote real de 3 reportes)

Algoritmo	Costo (m)	Estados expl.	Tiempo (ms)	¿Óptimo?	Ruta encontrada
BFS	1 820	16	-	Sí	BASE→R103→R102→R101
DFS	2 340	3	-	No	BASE→R101→R102→R103

Nota. Elaboración propia a partir del JSON exportado por el visualizador interactivo (Sección 4.2), generado el 10/07/2026 y verificado de forma independiente en Python.

Interpretación: DFS es mucho más rápido (3 estados explorados) pero se detiene en la primera ruta completa que encuentra, sin comparar alternativas: en este escenario resultó 28.6% más cara (2 340 m frente a 1 820 m) que el óptimo real. BFS, al expandir el árbol de búsqueda nivel por nivel sin detenerse hasta agotarlo, sí garantiza el costo mínimo -coincide exactamente con la fuerza bruta y con el óptimo exportado por el visualizador-, aunque para lograrlo tuvo que explorar más de 5 veces más estados que DFS (16 frente a 3).

6.3. Caso 2 - contraprueba con orden de exploración adverso

Para no basar la recomendación en un caso pequeño y favorable, se repite el experimento sobre el lote completo de 5 reportes ya formalizado en la Tabla 2 (R101 a R105), invirtiendo deliberadamente el orden en que DFS explora a sus vecinos. Esto expone con más claridad la diferencia entre ambas estrategias a medida que crece el tamaño del lote:

Tabla 5

Resultados comparativos de BFS y DFS - Caso 2 (lote de cinco reportes, orden adverso)

Algoritmo	Costo (m)	Estados expl.	Tiempo (ms)	¿Óptimo?	Ruta encontrada
BFS	4 250	326	0.35	Sí	Base→R101→R102→R103→R104→R105
DFS	5 650	5	0.01	No	Base→R105→R104→R103→R102→R101

Nota. Elaboración propia. Verificación ejecutada de forma independiente en Python, replicando la misma lógica del visualizador (Sección 5), sobre el grafo de cinco reportes de la Tabla 2.

Interpretación: con cinco reportes, DFS vuelve a ser drásticamente más rápido (5 estados) pero encuentra una ruta 32.9% más cara (5 650 m frente a 4 250 m): al invertir el orden de exploración, cambió por completo la ruta que “encontró primero”, confirmando que su resultado depende del orden de los datos, no de ningún criterio de costo. BFS, en cambio, volvió a coincidir exactamente con el óptimo de fuerza bruta, esta vez explorando 326 estados - 65 veces más que DFS. El patrón se repite y se acentúa: a medida que crece el lote, BFS sigue garantizando el óptimo, pero el costo de esa garantía (estados explorados) crece con el factorial del número de reportes.

6.4. El costo de garantizar el óptimo

Los dos casos muestran el mismo patrón: BFS garantiza el costo mínimo en ambas instancias (1 820 m y 4 250 m, exactamente el óptimo de fuerza bruta), pero paga ese resultado explorando entre 5 y 65 veces más estados que DFS. DFS, a cambio de terminar en unos pocos estados, no ofrece ninguna garantía: erró por 28.6% en el Caso 1 y por 32.9% en el Caso 2, y su resultado depende enteramente del orden en que el código itera los reportes pendientes - no de ningún criterio de costo o de negocio. Ninguno de los dos, tal como están implementados, aplica la regla de prioridad por gravedad formalizada en la Tabla 1: ambos pueden dejar una Emergencia para el final si así lo dicta el orden de los datos. Cerrar esa brecha -sin perder la garantía de optimalidad de BFS ni pagar su costo de exploración completo- es exactamente el problema que resuelve Backtracking con poda por costo y por prioridad, cuya implementación se plantea en la Sección 6.5 como siguiente paso del proyecto.

6.5. Recomendación final fundamentada

Algoritmo recomendado para esta entrega: BFS. La justificación combina dos criterios técnicos: (1) el negocio necesita minimizar metros reales de forma garantizada, no una respuesta que “a veces” es la mejor - eso descarta a DFS, cuyo resultado depende del orden arbitrario de los datos, como lo demuestra la contraprueba del Caso 2; (2) el tamaño real de un lote de reportes por técnico es pequeño (decenas, no miles), por lo que el mayor número de estados que explora BFS frente a DFS sigue siendo del orden de milisegundos y no representa un costo computacional relevante a esta escala. Queda pendiente, como siguiente paso del proyecto y no de esta actividad, implementar Backtracking con poda por costo y por prioridad de gravedad (formalizado en la Tabla 1 y en la Sección 5.5): es la única de las tres estrategias capaz de garantizar simultáneamente el óptimo y el cumplimiento de la regla de negocio, explorando una fracción de los estados que necesita BFS.

Trabajo futuro adicional (fuera del alcance de esta actividad): si Ojo Camba debe asignar reportes entre n técnicos simultáneamente, el problema se convierte en un Vehicle Routing Problem (VRP); backtracking exhaustivo dejaría de ser práctico por su crecimiento combinatorio ($n!$ por técnico) y correspondería migrar a heurísticas (vecino más cercano, algoritmos genéticos, OR-Tools de

Google) o a búsqueda informada con A* (Hart, Nilsson y Raphael, 1968), que combina costo real y heurística de distancia restante para explorar muchos menos nodos que un backtracking exhaustivo.

7. Auditoría de IA

Bitácora de uso y transparencia de IA, completada desde el primer día de trabajo, con registro de al menos un caso Descartado (fila 3) y evidencia de verificación humana independiente del código generado (fila 5):

Tabla 6

Bitácora de uso y transparencia de inteligencia artificial

Fecha	Fase	Herramienta	Prompt (resumen)	Sugerencia obtenida	Decisión	Verificación
Día 1	Selección del caso	Asistente de IA (chat)	Cómo extraer un grafo de OpenStreetMap para optimizar rutas	Sugirió OSMnx + NetworkX y explicó nodos/aristas/costo	Aceptado	Se contrastó con la documentación oficial de OSMnx
Día 1	Formalización	Asistente de IA (chat)	Cómo definir estado/meta/operadores para un técnico que visita varios reportes	Formalización con estado=(posición,visitados) ; meta=condición, no lugar	Aceptado	Verificado a mano con el caso de 2 reportes (Sección 5.1)
Día 2	Datos del caso	Asistente de IA (chat)	Cómo obtener tiempos de viaje reales entre puntos de Santa Cruz	Sugirió inicialmente una API de pago (Google Maps Distance Matrix)	Descartado	Se reemplazó por distancias estimadas por el equipo, documentadas como no oficiales
Día 3	Implementación	Asistente de IA (código)	Generar visualizador interactivo de BFS/DFS/backtracking	Primera versión resolvía llegar a UN reporte, no visitar todos	Modificado	Se exigió rehacer con estado=(posición , conjunto de visitados)
Día 3	Verificación	Auditoría propia (sin IA)	Cálculo manual del costo de las rutas legales del caso Base—E1—E2	N/A (verificación humana)	Validado	Coincide con la salida del código (Sección 5.1)
Día 4	Análisis	Asistente de IA (chat)	Por qué el botón de prioridad de Emergencia daba resultados inconsistentes	Detectó que la regla solo se aplicaba en un algoritmo, no en los tres	Corregido	Se unificó la regla y se validó con el caso B/E1/E2 a mano
Día 4	Análisis	Asistente de IA (código)	Ejecutar BFS/DFS/backtracking	Código funcional con contadores de nodos y tiempo	Aceptado y ejecutado	Salida real verificada línea

Fecha	Fase	Herramienta	Prompt (resumen)	Sugerencia obtenida	Decisión	Verificación
			sobre el lote de 5 reportes y comparar			por línea (Sección 5.2)

Nota. Elaboración propia. Registro cronológico de interacciones con IA durante el desarrollo de la actividad.

7.1. Auditoría crítica destacada

Situación: la primera versión del visualizador interactivo, generada con apoyo de IA, resolvía únicamente “llegar a un solo reporte” (búsqueda punto a punto), no “visitar todos los reportes del lote”, que es el problema real que enfrenta el técnico de Ojo Camba.

Decisión crítica tomada: se rechazó esa formalización y se exigió rehacerla con estado = (posición, conjunto de reportes visitados), meta = todos visitados, siguiendo el razonamiento del Paso 3 de este informe.

Justificación: aceptar sin cuestionar una simplificación que cambia el problema de fondo es exactamente el tipo de uso acrítico que la rúbrica institucional busca detectar en el Criterio 4. Verificar esta condición antes de aceptar la sugerencia - y, más adelante, diseñar la contraprueba de la Sección 5.3 para exponer que BFS/DFS pueden violar la regla de negocio - es lo que sostiene la validez técnica de este informe.

Un segundo hallazgo de auditoría: al pedir ayuda para calcular distancias reales entre puntos de Santa Cruz de la Sierra, la primera sugerencia de la IA fue usar la API de pago Google Maps Distance Matrix. Se descartó por introducir una dependencia de pago no declarada, y se reemplazó por (a) OpenStreetMap + Overpass API como opción gratuita para producción, o (b) estimaciones propias del equipo, documentadas explícitamente como no oficiales - la opción utilizada en este informe (Sección 1.2).

8. Conclusiones

El problema de ruteo del técnico de Ojo Camba se formaliza correctamente como un espacio de estados discreto, determinista, estático, totalmente observable y secuencial, resoluble mediante búsqueda simbólica sin necesidad de aprendizaje estadístico.

BFS garantiza el costo mínimo por búsqueda exhaustiva, pero explora entre 5 y 65 veces más estados que DFS en los dos casos probados; DFS es rápido pero sin ninguna garantía de optimalidad, y su resultado depende del orden arbitrario de los datos, no de ningún criterio de costo, como lo demuestra la contraprueba del Caso 2 (Sección 6.3).

Ninguno de los dos algoritmos implementados aplica la restricción de prioridad por gravedad formalizada en la Tabla 1; cerrar esa brecha sin perder la garantía de optimalidad de BFS es el objetivo de Backtracking con poda combinada, recomendado como siguiente paso del proyecto (Sección 6.5).

El “costo de garantizar el óptimo” (Sección 6.4) es una métrica reutilizable para que Ojo Camba dimensione, lote a lote, cuántos estados de más debe explorar BFS para no arriesgarse a una ruta subóptima - información valiosa tanto para decisiones técnicas como de negocio.

9. Referencias

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100.

Newell, A., & Simon, H. A. (1956). The logic theory machine: A complex information processing system. *IRE Transactions on Information Theory*, 2(3), 61–79.

Newell, A., Shaw, J. C., & Simon, H. A. (1959). Report on a general problem-solving program. *Proceedings of the International Conference on Information Processing*, 256–264.

Ojo Camba. (2026). Ojo Camba [Repositorio de código]. GitHub.

<https://github.com/jhoel0521/ojo-camba>